MINI PROJECT REPORT

GAMING & SIMULATION ENGINES USING OOP IN C++

NEXUS ARENA — Game Logic & Physics Simulation

Submitted by	Muskan Gupta
UID	25bcd10069
Section	25bcd-2a
Submitted to	Mr Jitendra Kumar

GitHub Repository

https://github.com/muskangupta434343-beep/gaming.oops

Academic mini project • C++17 • Object-Oriented Programming

Contents & Project Overview

Page	Section
1	Cover page
2	Contents & project overview
3	1. Introduction
4	2. Objectives / Problem Statement
5	3. Modules and system design
6	Tools and technologies used
7	OOP concepts and class design
8	Code — classes and game engine
9	Code — simulation engine and main
10	Program output screenshot
11	Game-style visual output screenshot
12	Testing, conclusion and repository

Project overview

Nexus Arena is an educational project demonstrating how object-oriented programming organises a small game and a simple physics simulation. The game models a player and enemies, tracks health, energy and score, and runs a short sequence of turns. The simulation updates position and velocity over fixed time steps.

The source is a C++17 console program. A separate game-style visual is included to make the demo state easy to understand; it is a designed presentation of the program state, not a live interactive graphics engine.

1. Introduction

Gaming and simulation engines manage state, rules, events and updates over time. A game engine coordinates player actions, enemy behaviour, health, scoring and levels. A simulation engine models a process—such as motion—by repeatedly applying rules over small time intervals.

C++ is widely used for performance-oriented applications and supports object-oriented programming (OOP). OOP groups data and related behaviour into classes. This makes a project easier to extend: a new character type can reuse a base class, while a new simulation model can be added without rewriting the game loop.

This project combines two learning examples:

• **Game demonstration:** Muskan encounters a Cyber Drone and a Mecha Beast. Attacks affect enemy health and defeated enemies can award points.
• **Physics demonstration:** a body begins with a position and velocity; acceleration changes velocity and the engine updates position in discrete time steps.

The project is a teaching prototype, not a commercial engine. It focuses on internal logic and state transitions rather than real-time graphics, audio, collision detection or an asset pipeline. The visual output page presents the game state in a HUD-style layout.

Why this topic matters

The same patterns appear in larger systems: game objects own state, engines coordinate updates, and simulation steps make behaviour repeatable. Practising these ideas builds a foundation for graphics libraries, physics engines and interactive applications.

2. Objectives / Problem Statement

Problem statement

Game logic written as one long sequence becomes difficult to maintain as more characters, enemy types, rules or simulation variables are added. The problem is to create a compact, understandable program that separates character data, player-specific features, enemy behaviour, game coordination and physics calculations.

Objectives

• Represent game entities with classes and objects.

• Use encapsulation for health, score, energy and simulation variables.

• Use inheritance so Player and Enemy reuse Character.

• Demonstrate methods and virtual functions for reusable behaviour.

• Implement turns, attacks, health changes and score updates.

• Implement a discrete-time physics example with position, velocity and acceleration.

• Provide readable program output and a game-style visual.

• Keep the source portable and compatible with C++17.

Scope and assumptions

The demo runs a predetermined sequence instead of accepting keyboard input. Damage values and enemy health are fixed for repeatable output. The physics example uses constant acceleration and a fixed time step. It is a learning example rather than a high-accuracy physics solver.

Expected result

Execution prints a title, initial status, combat actions, final score and a physics table. The same source and compiler should produce consistent output.

3. Modules and System Design

The project is divided into five logical modules, each with a focused responsibility.

Module	Responsibility
Character	Base class for name, health, maximum health and damage handling.
Player	Extends Character with score, energy and a special move.
Enemy	Extends Character with attack power and turn behaviour.
GameEngine	Owns player and enemies; displays status and coordinates rounds.
SimulationEngine	Updates position and velocity over fixed time steps.

Execution flow

• main() creates GameEngine with the player name Muskan.

• GameEngine initializes a player and two enemies.

• runDemo() prints status and executes the combat rounds.

• Attacks reduce target health; a defeated enemy can award score.

• SimulationEngine runs after combat and prints time-step values.

Design note: GameEngine coordinates the game, while each character stores its own state. This separation keeps responsibilities clear.

Tools and Technologies Used

Technology	Purpose
C++17	Main language; classes, inheritance, vectors, strings and standard I/O.
C++ Standard Library	vector stores enemies; algorithm utilities and iomanip format values.
g++ or compatible compiler	Builds the source. Visual Studio, clang++ or another C++17 compiler may also be used.
Terminal / IDE	Build and run through VS Code, Visual Studio, Code::Blocks or a terminal.
PNG screenshots and PDF	Document captured console output and provide a game-style visual.
GitHub	Source hosting location supplied by the student.

How to compile and run

```
g++ -std=c++17 -O2 gaming_simulation_engine.cpp -o nexus_arena
./nexus_arena
```

On Windows, run nexus_arena.exe from Command Prompt or use the IDE's run command. Compiler installation details may change the exact command.

GitHub repository

Open the supplied GitHub repository. The URL is included as provided; repository contents and permissions are not independently verified here.

OOP Concepts and Class Design

Encapsulation

Character stores name, health and maxHealth as protected fields. Player-specific score and energy are private. Public methods such as getHealth(), addScore() and takeDamage() provide controlled access to state.

Inheritance

Player and Enemy inherit from Character. They reuse common health management and name handling while adding their own features. This avoids duplicating the base behaviour.

Polymorphism

The base class declares attack() and takeDamage() as virtual. Derived classes can customise these methods in future versions. This extension point could support armour, shields, bosses or different damage rules.

Abstraction

GameEngine hides combat sequencing behind runDemo() and showStatus(). SimulationEngine exposes runPhysicsDemo() rather than making main() manage every update.

Class relationship overview

Responsibility mapping

- Character: identity and health.
- Player: score, energy and special move.
- Enemy: attack power and turn logic.
- GameEngine: combat state and round coordination.
- SimulationEngine: time step, position, velocity and acceleration.

Code — Character, Player and Enemy

The following core classes show the OOP structure. The complete source file is supplied separately.

```
class Character {
protected:
    string name;
    int health, maxHealth;
public:
    Character(string n, int hp)
        : name(n), health(hp), maxHealth(hp) {}
    virtual ~Character() = default;
    virtual void attack(Character& target, int damage) {
        target.takeDamage(damage);
    }
    virtual void takeDamage(int damage) {
        health = max(0, health - max(0, damage));
    }
    bool isAlive() const { return health > 0; }
    int getHealth() const { return health; }
    int getMaxHealth() const { return maxHealth; }
    string getName() const { return name; }
};

class Player : public Character {
    int score, energy;
public:
    Player(string n) : Character(n, 100), score(0), energy(3) {}
    void useSpecialMove(Character& target) {
        if (energy > 0 && isAlive() && target.isAlive()) {
            cout << name << " uses POWER BLAST!\\n";
            attack(target, 25);
            --energy;
        } else cout << "Power Blast unavailable.\\n";
    }
    void addScore(int points) { score += points; }
    int getScore() const { return score; }
    int getEnergy() const { return energy; }
};

class Enemy : public Character {
    int attackPower;
public:
    Enemy(string n, int hp, int power)
        : Character(n, hp), attackPower(power) {}
    void takeTurn(Player& player) {
        if (isAlive() && player.isAlive()) {
            cout << name << " attacks for "
                 << attackPower << " damage.\\n";
            attack(player, attackPower);
        }
    }
};
```

Implementation notes

The constructor initialises each object. takeDamage() clamps health at zero and prevents negative damage. isAlive() checks whether an entity can act. The special move consumes energy only when the move is available and both participants are alive.

The Character reference passed to attack() lets the same method affect different character objects. Damage handling stays centralised rather than duplicated in each class.

Code — Engines and main()

This excerpt shows the coordinator classes and the discrete-time physics demonstration.

```
class GameEngine {
    Player player;
    vector<Enemy> enemies;
public:
    GameEngine(string playerName) : player(playerName) {
        enemies.emplace_back("Cyber Drone", 45, 12);
        enemies.emplace_back("Mecha Beast", 60, 15);
    }
    void showStatus() const {
        cout << "\\n=== GAME STATUS ===\\n";
        cout << player.getName() << " HP: "
             << player.getHealth() << "/"
             << player.getMaxHealth()
             << " | Score: " << player.getScore()
             << " | Energy: " << player.getEnergy() << "\\n";
        for (const auto& e : enemies)
            cout << e.getName() << " HP: "
                 << e.getHealth() << "/" << e.getMaxHealth() << "\\n";
    }
    void runDemo(); // coordinates combat rounds
};

class SimulationEngine {
    double timeStep;
public:
    explicit SimulationEngine(double dt) : timeStep(dt) {}
    void runPhysicsDemo() const {
        double position=0.0, velocity=4.0, acceleration=2.0;
        cout << "Time(s)  Position(m)  Velocity(m/s)\\n";
        for (int step=0; step<=4; ++step) {
            cout << fixed << setprecision(1)
                 << setw(4) << step*timeStep << "     "
                 << setw(6) << position << "        "
                 << setw(5) << velocity << "\\n";
            velocity += acceleration*timeStep;
            position += velocity*timeStep;
        }
    }
};

int main() {
    GameEngine game("Muskan");
    game.runDemo();
    SimulationEngine simulation(0.5);
    simulation.runPhysicsDemo();
    return 0;
}
```

Physics update

The time step is 0.5 seconds and acceleration is 2.0 m/s². At each step, the program prints current values, updates velocity by acceleration × time step, then updates position using the updated velocity. This simple sequence demonstrates repeated state updates rather than a high-accuracy physical integrator.

The complete source file should be compiled directly; the report excerpts are shortened for readability.

4. Code with Outputs — Program Screenshot

The image below is a screenshot-style rendering of the program's captured standard output. It shows the title, combat actions, status values and physics table, rather than presenting the output only as plain text.

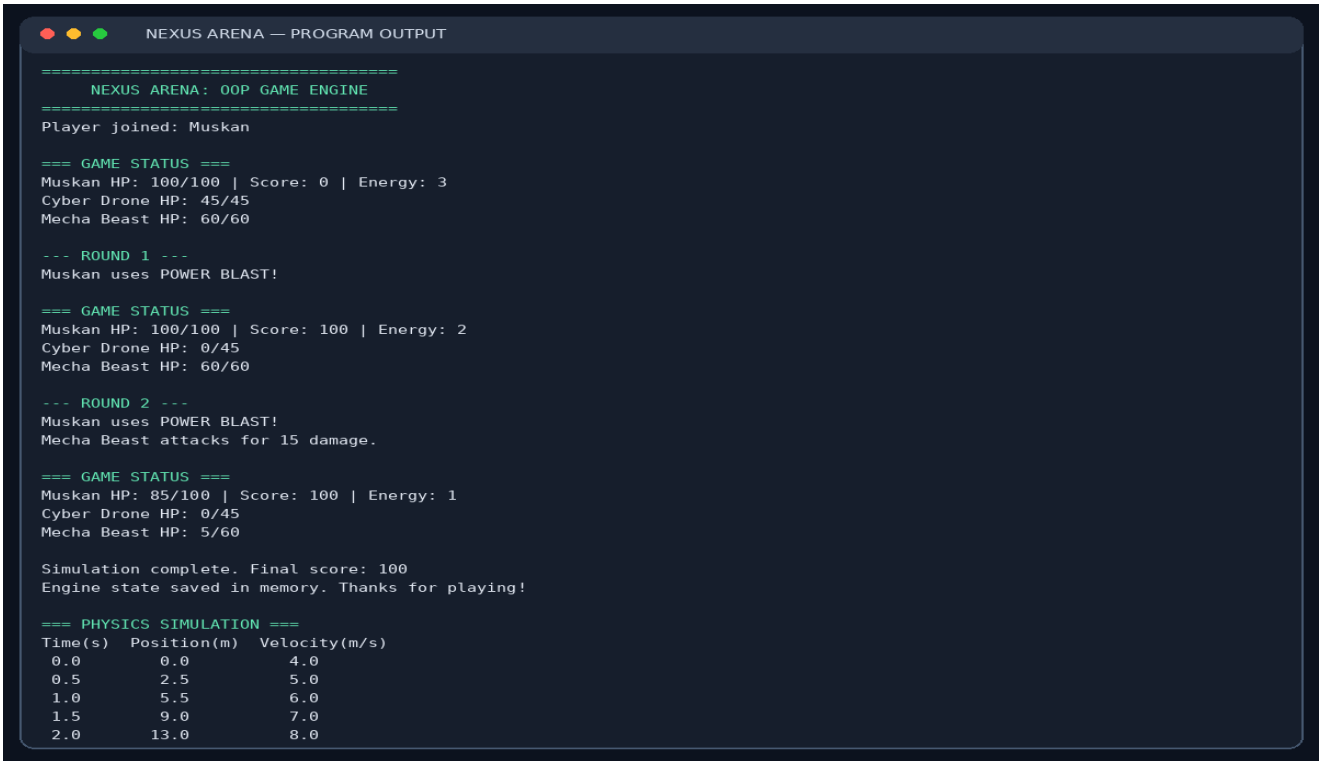


Figure 1. Console output captured from the C++ demo run. Open the companion PNG for a larger view.

Visible behaviours

- Player starts with 100 health and 3 energy units.
- A special move spends energy and damages an enemy.
- Enemy turns reduce player health.
- Defeating an enemy awards score.
- Physics values are printed at 0.5-second intervals.

Game-Style Visual Output

This visual presents the demo as a game HUD with a player avatar, enemy, health bar, score, energy and round status. It is a designed visual representation of the demo state, not a screenshot from a separate graphics engine; the C++ application itself is a console program.

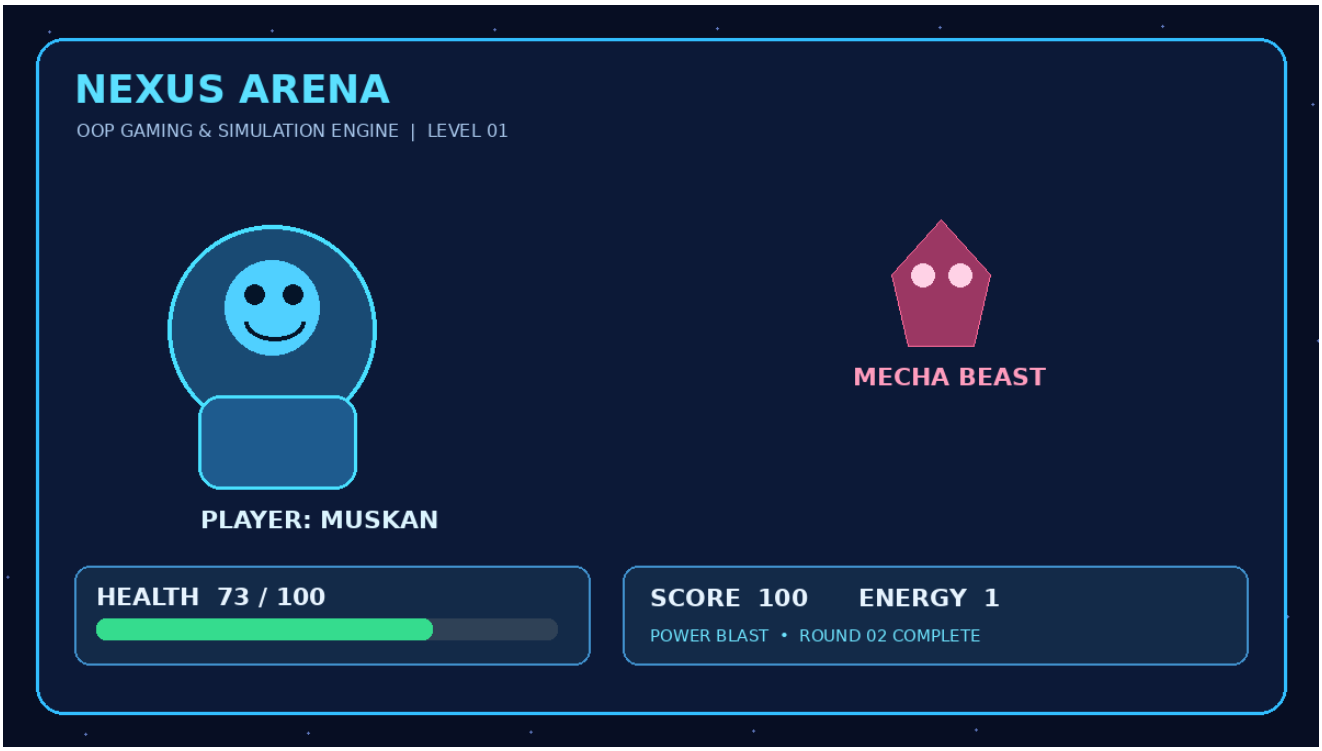


Figure 2. Nexus Arena game-style visual representing the demo state.

Possible graphical extension

A future version could connect the same Character and GameEngine logic to SFML or SDL. A render loop could draw sprites and health bars, read keyboard events, update the engine each frame and play sound effects. Separating logic from rendering makes that extension more manageable.

Testing, Conclusion and Repository

Testing checklist

Test	Expected result
Initialisation	Player health is 100; energy is 3; both enemies start at full health.
Special move	Enemy health decreases by 25 and energy decreases by one.
Enemy turn	Player health decreases by the enemy's attack power.
Health boundary	Health does not drop below zero.
Score update	A defeated enemy awards the configured score.
Physics steps	Five rows print from 0.0 to 2.0 seconds.

5. Conclusion

This mini project demonstrates how OOP structures a small game and a basic physics simulation in C++. Character, Player and Enemy model related entities, while GameEngine coordinates combat and SimulationEngine performs fixed-step updates. The project applies encapsulation, inheritance, abstraction and a polymorphism-ready interface in a compact example.

The program could be extended with keyboard input, random enemy AI, levels, inventory, collision detection, graphics and sound. Its current limitation is that combat is a fixed demonstration and the game-style image is a visual mock-up rather than an interactive rendered game.

GitHub repository

https://github.com/muskangupta434343-beep/gaming.oops

Submitted by: Muskan Gupta | UID: 25bcd10069 | Section: 25bcd-2a
Submitted to: Mr Jitendra Kumar

Companion source file: gaming_simulation_engine.cpp